

Assignment 1: Implementing a Basic Shell

MTL458: Operating Systems

Semester I, 2026-2027

Introduction

In this assignment, you'll build a command shell of your own, a stripped-down version of bash that reads what you type, runs it, and manages everything in between: connecting commands with pipes, sending their input and output to files, and deciding whether to keep going based on how the last command turned out. Under the hood, every command you run means forking a child process, using `exec` to load a program into it, and waiting for it to finish, the same building blocks real shells, container runtimes, and job schedulers are built on.

Environment and Build Requirements

- **Language:** C, compiled with GCC 11.1 or later, under the C17 standard. Do not use the `readline` library, or install or link against any other third-party library.
- Your shell must print the prompt `MTL458 >` (note the trailing space) before reading each line of input, and must keep looping, reading, parsing, and executing, until it reads the `exit` command.

Assumed Working Directory

Your shell will be started with the following directory as its current working directory, and this is also where all relative paths in the examples below resolve from:

```
/home/user/  
  notes.txt  
  data.txt  
  records/  
    entry.txt  
  greet.sh
```

File contents (every file ends with a trailing newline):

- `notes.txt`: `alpha beta`
- `data.txt`: `gamma delta`
- `records/entry.txt`: `gamma gamma gamma`
- `greet.sh`: `echo "Greetings from script"`

Reading and Executing Commands (15 points)

For any command that is not one of the built-ins described below, your shell must fork a child process and use `execvp` (or another member of the `exec` family) to run it, then wait for it to finish before printing the next prompt.

Input:

```
echo shell test
```

Expected Output:

```
shell test
```

Tokenization rules:

- Arguments are separated by whitespace. Runs of multiple spaces between arguments are equivalent to a single space and must not produce empty arguments.
- A line will contain no more than 100 arguments and no more than 2048 characters in total.
- None of the special characters `|`, `&`, `;`, `<`, `>` will appear inside an argument, a filename, or a directory name in any test input. You do not need to handle escaping or quoting of these characters.
- A bare Enter keypress (an empty line) produces no output at all and simply reprints the prompt.

If a command cannot be run at all, because it does not match a built-in and is not found via `execvp`'s `PATH` search, or because `fork()` fails, print `Invalid Command` on its own line. This is the only error message your shell ever prints; built-in misuse (see `cd` and `history` below) also produces it.

Input:

```
nosuchcommand
```

Expected Output:

```
Invalid Command
```

For an external command that runs successfully but itself reports an error, such as wrong flags, a file it cannot find, or a pattern it cannot match, you may either print `Invalid Command` yourself or let the command's own error message through unmodified; both are accepted. Your shell must never crash, and must always return to prompting for the next command.

Built-in Commands

Three commands must be implemented directly in your shell rather than executed as external programs: `cd`, `history`, and `exit`.

`cd` (15 points)

- `cd`: Change to the home directory. Prints nothing on success.
- `cd ~`: Same as bare `cd`.
- `cd -`: Switch to the previous working directory, and print the resulting absolute path. This is the one `cd` form that produces output, matching `bash`.

- `cd .`: Stay in the current directory. A no-op, but it must be accepted.
- `cd ..`: Move to the parent directory.
- `cd path`: Move to `path`, which may be relative (for example `records` or `../sibling`) or absolute.

If the target directory does not exist, print `Invalid Command` and leave the current working directory unchanged. `cd` never receives more than one path argument in a valid test; if it receives zero or more than one argument, other than the special forms above, that is also `Invalid Command`.

Input:

```
cd records
pwd
```

Expected Output:

```
/home/user/records
```

(`pwd` itself is not a built-in. It is the real `pwd` executable, run like any other external command, and it is how correctness of `cd` is verified throughout this handout and in the test suite.)

history and history n (12 points)

`history` prints every command entered so far, including the `history` command itself. `history n` prints only the last `n` commands. The maximum number of commands stored is 2048.

- An entry is the raw line exactly as typed, including any `;`, `&&`, `||`, redirection, or pipe operators it contains.
- A line that fails, whether an unrecognized command or a built-in used incorrectly, is still recorded, verbatim, exactly as it would be if it had succeeded.
- A bare Enter (empty line) is never recorded.
- `history 0` will not appear in any test input.

Input:

```
pwd
echo hi
history
```

Expected Output:

```
/home/user
hi
pwd
echo hi
history
```

exit

Terminates the shell process. No other key combination, such as `Ctrl+C`, is required or tested to terminate the shell. `exit` is the only documented way out.

Pipes (18 points)

Your shell must support chaining up to two pipe (`|`) operators, that is, pipelines of up to three commands, where the standard output of each command becomes the standard input of the next. The `|` operator is always surrounded by spaces in test input. Neither `cd` nor `history` will appear as a stage in a pipeline.

Input:

```
cat data.txt | grep gamma
```

Expected Output:

```
gamma delta
```

Input:

```
cat records/entry.txt | grep gamma | wc -l
```

Expected Output:

```
1
```

If a stage in the pipeline names a command that cannot be found, print `Invalid Command` for that stage and let the remaining stage(s) still run. A broken stage does not abort the whole pipeline, mirroring how a real shell handles it: the failing stage's own process reports the problem, and its siblings in the pipeline are unaffected.

Input:

```
madeupcmd | grep gamma
```

Expected Output:

```
Invalid Command
```

Make sure every pipe file descriptor your process does not need is closed in every process, parent and each child, before that process blocks on I/O. A shell that forgets to close an unused write end of a pipe will hang forever waiting for an EOF that never comes.

I/O Redirection (20 points)

Your shell must support the redirection operators `<` (redirect standard input from a file), `>` (redirect standard output to a file, truncating it), and `>>` (redirect standard output to a file, appending to it). These operators may or may not be surrounded by spaces in test input.

Input:

```
echo greetings > note_out.txt
cat note_out.txt
```

Expected Output:

```
greetings
```

A command may carry at most one input redirection and at most one output redirection (> or >>, not both) in the same line: never two of the same kind, and never combined with a pipe. When both are present, they may appear in either order relative to each other:

Input:

```
sort < data.txt > sorted.txt
cat sorted.txt
```

Expected Output:

```
gamma delta
```

Input:

```
wc -l > count.txt < notes.txt
cat count.txt
```

Expected Output:

```
1
```

Both orderings above must produce identical results. > and >> always create the target file if it does not already exist; a command that produces no output still creates an empty file when its output is redirected.

Command Separators: ;, &&, || (20 points)

A single input line may contain multiple commands joined by ;, &&, or ||. These operators may or may not be surrounded by spaces, follow bash's usual left-to-right precedence, and will never appear inside parentheses. Parentheses will not appear in any test input at all.

- `cmd1 ; cmd2`: run `cmd1`, then run `cmd2` regardless of whether `cmd1` succeeded.
- `cmd1 && cmd2`: run `cmd2` only if `cmd1` succeeded.
- `cmd1 || cmd2`: run `cmd2` only if `cmd1` failed.

“Succeeded” means the child process exited with status 0, checked via `waitpid` and `WEXITSTATUS`, not just that the command was found. A command that is not found at all never produces a successful exit status, so it always counts as a failure for `&&` and `||`, in addition to printing `Invalid Command`.

Input:

```
echo first ; echo second
```

Expected Output:

```
first
second
```

Input:

```
echo ready && echo go
```

Expected Output:

```
ready
go
```

Input:

```
madeupcmd || echo recovered
```

Expected Output:

```
Invalid Command
recovered
```

Submission

- Submit through Gradescope. Your file must be named `entrynumber_a1.c`, where `entrynumber` is your IIT Delhi entry number in lowercase.
- Ensure your code is well-commented and follows best practices for readability and maintainability.
- All submissions will be strictly checked for plagiarism, including comparisons with submissions from previous years as well as AI-generated content. Any instance of plagiarism will result in a score of zero for the assignment.

Good luck, and happy coding!